

QCon
全球软件开发大会

生成式UI：AI交互新模式探索

林瑞虹

云软件体验技术团队（华为）/高级开发工程师

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 生成式UI——开启AI交互新模式

02 生成式UI 原理与基于场景落地扩展

03 协议对比与标准化争论

01 生成式UI

——开启AI交互新模式

改善文本对话体验，可视化直观清晰

传统AI交互多为高密度长文本流，理解成本高。生成式UI能自动将文本转化为柱状图、饼图等可视化图表，大幅提升信息获取效率。



Markdown效果



使用生成式UI



信息降维

将复杂数据转化为视觉语言



直观高效

一眼识别关键趋势与结论



自动生成

无需手动操作，AI 实时渲染



优化多轮信息交互， 贴合业务降低门槛

在多轮对话中，用户需要反复输入指令来确认选项，过程繁琐。生成式UI可以直接生成可视化的选择界面，用户通过点击即可完成操作，大大简化了交互流程，降低了用户的使用门槛，让AI交互更贴合实际业务场景

好的，我们来一步步确认您的奶茶订单：

第一步：选择奶茶类型 请从以下选项中选择您想要的奶茶：

1. 珍珠奶茶 (15元)
2. 芋圆奶茶 (16元)
3. 芝士奶盖 (18元)
4. 水果茶 (20元)
5. 柠檬茶 (12元)

您想喝哪种奶茶呢？

水果茶

好的，已选择：水果茶 (20元)

第二步：选择甜度 请选择甜度：

1. 无糖
2. 微糖
3. 半糖
4. 标准糖
5. 多糖

您想要什么甜度呢？

标准

好的，已选择：标准糖

第三步：选择冰度 请选择冰度：

1. 去冰
2. 少冰
3. 正常冰
4. 多冰

您想要什么冰度呢？

3

好的，已选择：正常冰

第四步：选择杯型 请选择杯型：

1. 中杯 (原价)
2. 大杯 (+20%)
3. 超大杯 (+50%)

您想要什么杯型呢？

大

好的，已选择：大杯

第五步：选择加料 (可多选) 请选择您想要加的配料 (每份2元)：

1. 珍珠
2. 芋圆
3. 布丁
4. 椰果
5. 红豆

您想要加什么配料呢？ (可以选多个，用逗号分隔，如：1,3 或直接说“不需要”)

不要

好的，不加配料。

第六步：选择数量 请选择购买数量 (1-10杯)：

您需要几杯呢？

1

好的，数量：1杯

第七步：添加备注 您有什么特殊要求吗？ (比如：少冰、去珍珠等)

请输入备注 (如果没有特殊要求，请说“无”)：

无

好的，没有特殊备注。

您的奶茶订单已确认：

- 奶茶类型：水果茶
- 甜度：标准糖
- 冰度：正常冰
- 杯型：大杯
- 加料：无
- 数量：1杯
- 备注：无

总价计算：

- 水果茶基础价：20元
- 大杯加价：20元 × 20% = 4元
- 加料费用：0元
- 总计：24元

是否确认提交订单？ (回复“确认”或“取消修改”)



文本交互
繁琐指令输入

点一杯奶茶

* 选择奶茶

水果茶

甜度 冰度

标准糖 正常冰

杯型 数量

中杯 大杯 超大杯 1

加料

☐ 珍珠 ☐ 芋圆 ☐ 布丁 ☐ 椰果 ☐ 红豆

备注

请输入特殊要求 (如：少冰、去珍珠等)

预估价格：24元 (价格根据奶茶类型、加料和杯型计算)

提交订单 重置



可视化UI
一键点击操作

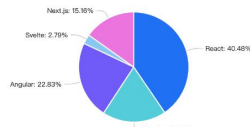
千人千面实时渲染，告别“预制”轻松调整

生成式UI打破传统“预制”限制，基于用户偏好实时渲染内容。无论是严谨的专业报告，还是活泼的演示文稿，只需简单指令即可一键切换文案、配色与布局，实现真正的个性化体验。

前端框架发展趋势分析报告 (2024)

本报告基于State of JS 2023、npm trends及行业分析数据，旨在客观呈现当前主流前端框架的发展态势与关键技术趋势。

市场占有 增长趋势 核心结论



框架名称	市场占有 (%)	年增长 (%)	核心特性	框架描述
React	40.6	2.1	组件化、单向数据流、Hooks	由Facebook维护，采用JSX语法构建UI，生态庞大，社区活跃。
Vue.js	18.8	1.5	响应式系统、组合式API、SFC	渐进式框架，易于上手，在中国市场占有重要地位。
Angular	22.9	0.8	TypeScript、依赖注入、MVC	企业级全栈框架，由Google支持，TypeScript已成为大型项目的标配。
Svelte	2.8	15.7	无运行时、反应式声明、编译优化	编译时框架，无虚拟DOM，打包体积小，性能优异。
Next.js	15.2	25.3	服务端渲染、文件路由、SSR/SSG、专注于生产力和性能。	基于React的元框架，支持SSR/SSG，专注于生产力和性能。

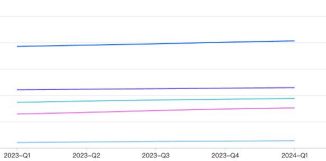
如需获取更详细数据或定制分析报告，请联系我们。

获取完整报告

前端框架发展趋势分析报告 (2024)

本报告基于State of JS 2023、npm trends及行业分析数据，旨在客观呈现当前主流前端框架的发展态势与关键技术趋势。

市场占有 增长趋势 核心结论



前端框架发展趋势分析报告 (2024)

本报告基于State of JS 2023、npm trends及行业分析数据，旨在客观呈现当前主流前端框架的发展态势与关键技术趋势。

市场占有 增长趋势 核心结论

主要发展趋势

- 元框架崛起：Next.js (React)、Nuxt.js (Vue) 等提供全栈能力的框架增长显著，简化了部署、路由和性能优化。
- 编译时优化：Svelte、SolidJS 等框架通过编译时技术减少运行时开销，提升应用性能与用户体验。
- TypeScript普及：Angular 内置支持，React、Vue 官方推荐，TypeScript已成为大型项目的标配。
- 服务端组件与边缘渲染：React Server Components、Astro 等框架加速边缘与服务器端渲染，优化首屏性能。
- 低代码/无代码集成：前端框架与可视化构建工具结合，提升开发效率，降低技术门槛。

选型建议

- 大型企业应用：推荐 Angular (全栈方案) 或 React/Next.js (生态丰富)。
- 快速原型与中小型项目：Vue.js 或 Svelte 学习曲线平缓，开发效率高。
- 性能敏感型应用：考虑 Svelte、SolidJS 或使用 Next.js 进行服务端渲染。
- 内容型网站：Astro、Next.js (SSG) 等静态站点生成器是优选。

数据来源：State of JS 2023, npm trends, GitHub Octoverse, 报告生成时间：2024年5月。

如需获取更详细数据或定制分析报告，请联系我们。

获取完整报告

严谨专业

前端框架风云榜 2024

谁才是你心中的 C 位？一起来聊聊~

人气爆棚 潮流风向 我该选谁？

1. **React**
老大哥，生态巨无霸
Facebook 亲儿子，JSX 写起来像写 HTML，虚拟 DOM 玩得溜。
Hooks 一出，函数组件原地起飞~
热度 🔥🔥🔥

冷知识：据说每10个前端，7个写过 React

2. **Vue.js**
渐进式的优雅
尤大出品，必属精品！单文件组件 (vue) 用起来超舒服，中文文档友好到哭，国内开发者最爱没有之一！
热度 🔥🔥🔥

冷知识：真香定律在 Vue 3 组合式 API 上再次应验

3. **Angular**
企业级全家桶
Google 大佬背书，TypeScript 原生支持，自带路由、HTTP、表单验证... 开箱即用，适合大型团队开展。
热度 🔥🔥

冷知识：学 Angular = 学了一套完整的开发哲学

4. **Svelte**
魔法编译，先组胜有招
这有虚拟 DOM！编译时搞定一切，打包体积小得离谱，性能杠杠的！
写起来像写原生 JS，但更爽！
热度 🔥🔥🔥

冷知识：粉丝口号：“Svelte 是未来！” (至少他们这么认为)

5. **Next.js**
React 的终极进化形态
文件即路由，服务端渲染、图片自动优化... Vercel 出品，让 React 开发变得 so easy！妈妈再也不用担心我的 SEO 和性能了。
热度 🔥🔥

冷知识：“npm create-next-app”可能是最香的初始化命令

前端框架风云榜 2024

谁才是你心中的 C 位？一起来聊聊~

人气爆棚 潮流风向 我该选谁？

选择困难症？看这里！

入门入门
Vue 3 > Svelte > React，从最简单易学，快乐学习最重要！别一开始就挑战地狱难度。
性能狂魔福音
Svelte/SolidJS 编译时魔法，Check 的魔法渲染，Astro 的岛式架构，发量素到前仆后继！

如果你有团队！技术选型就像谈恋爱，也很重要，别盲目追求保持好奇，你就是最靓的仔！

来聊聊你的想法！
哪个框架？或者有更酷的选型分享？

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

发起我的想法！

前端框架风云榜 2024

谁才是你心中的 C 位？一起来聊聊~

人气爆棚 潮流风向 我该选谁？

2024 年前端圈都在玩啥？

元框架 (Next.js, Nuxt, SvelteKit) 是根本答案

服务器组件 & 边缘渲染：让浏览器都会，服务器多干点

TypeScript：从“可选”变成“真香”再到“必备”

构建工具链卷疯了：Vite > Webpack, Turbopack > ?

低代码/无代码：让前端开发更傻瓜，也让前端更危险了

其他/框架：10%

传统三大框架：10%

元框架 (Next/Nuxt等)

编译时框架 (Svelte/SolidJS)

传统三大框架 (Vue/React/Angular)

其他/框架

数据来源：State of JS 2023 + 小编的脑补

活泼生动

OpenTiny GenUI SDK 多轮对话案例



GenUI Playground

请输入您的问题~

内容由AI生成，仅供参考

02

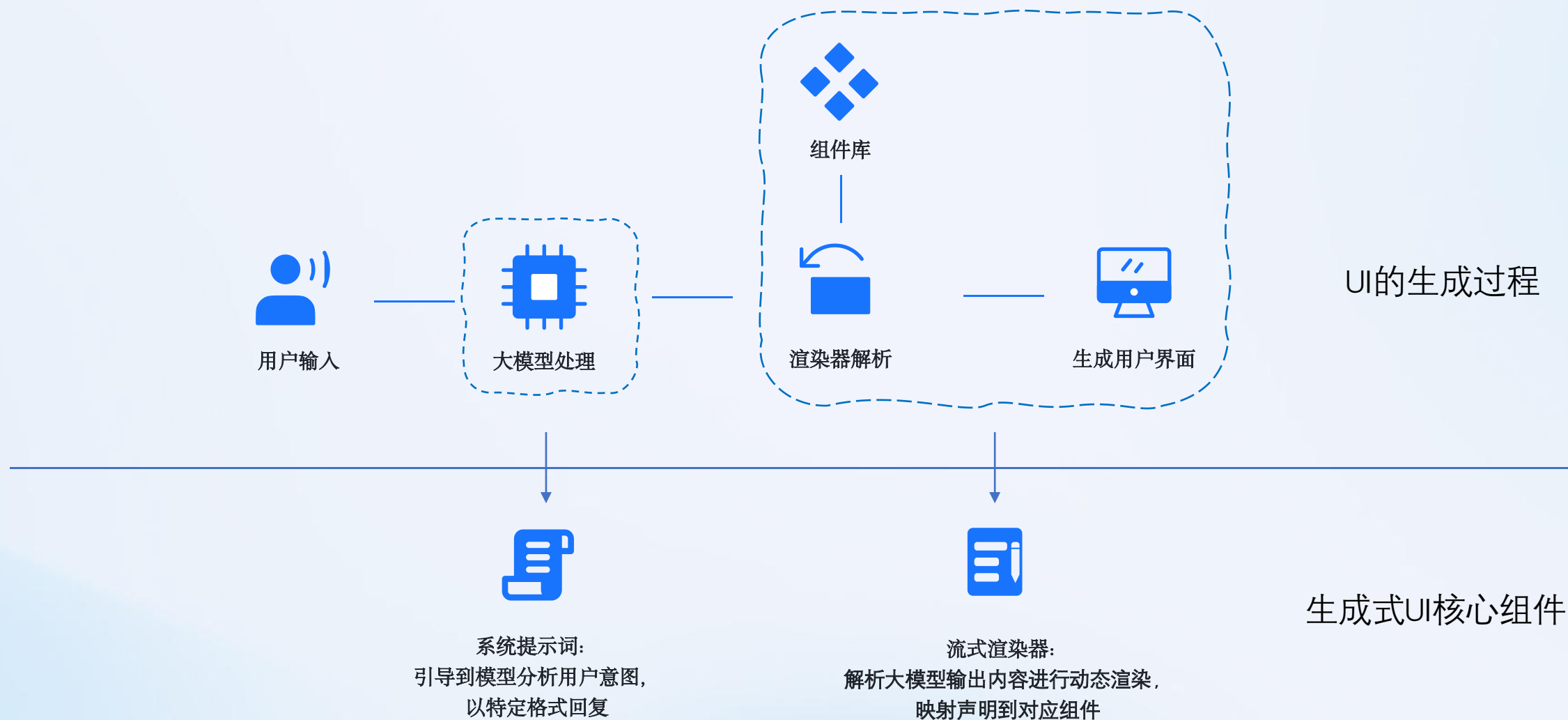
原理与场景落地

02

原理与场景落地

1 生成式UI实现原理

UI是如何生成的?



核心原理机制解析



结构化输出

大模型输出结构化的UI描述信息，而非纯文本，为前端渲染提供清晰指令。



流式增量渲染

前端接收到部分结构信息后即可开始渲染，无需等待完整数据，大幅提升用户体验。



缓冲保护区

建立数据校验机制，过滤掉不完整或错误的信息，确保界面渲染的稳定性和正确性。

● 原理：始于结构化输出

大模型结构化输出三种形式



指定格式

Chat Request (OpenAI) :
{..., response_format: {
 type: 'json_schema',
 json_schema: { schema: { xxx }
}}



工具调用

Chat Request:
{..., tools: [{ ..., parameters: {
 type: 'object',
 properties: { xxx }
}}, ...]}



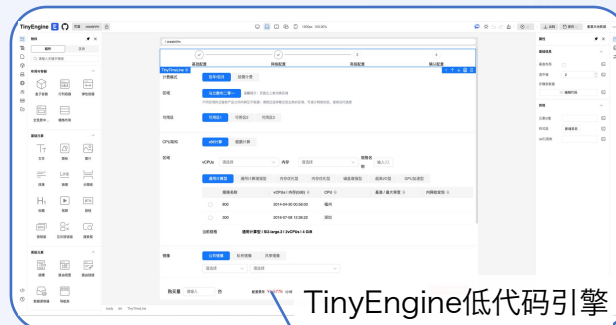
系统提示词
约束

System prompt:
You are a helpful assistant. ...
Please response in following format:
xxx

相同点：可以使用 JSON Schema 描述结构化输出

基于JSON的声明式UI

低代码平台



TinyEngine低代码引擎

页面协议

```
{  
  "componentName": "Page",  
  "state": {},  
  "children": [  
    {  
      "componentName": "div",  
      "props": {  
        // ...  
      },  
      // ...  
    },  
    // ...  
  ]  
}
```



渲染器

+



UI描述协议

生成式UI

采用低代码协议，通过声明式UI的结构化输出和渲染，实现AI自主展示UI界面

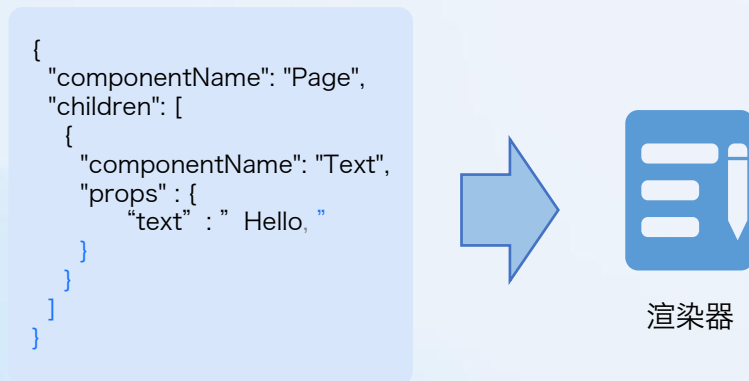


原理：流式增量渲染

大模型输出特点：流式

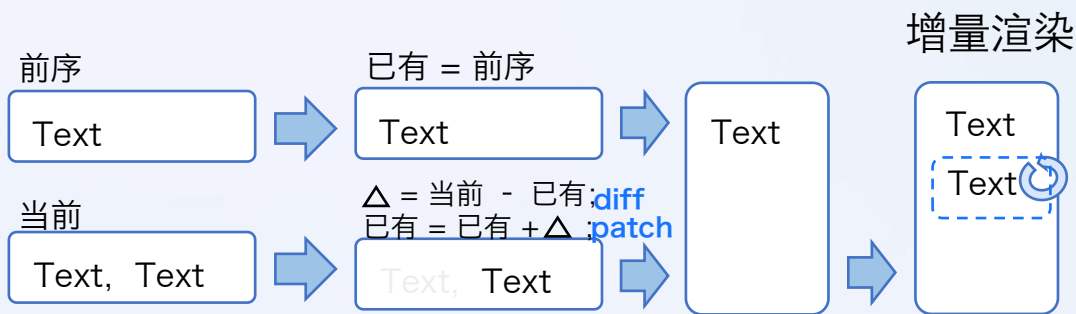
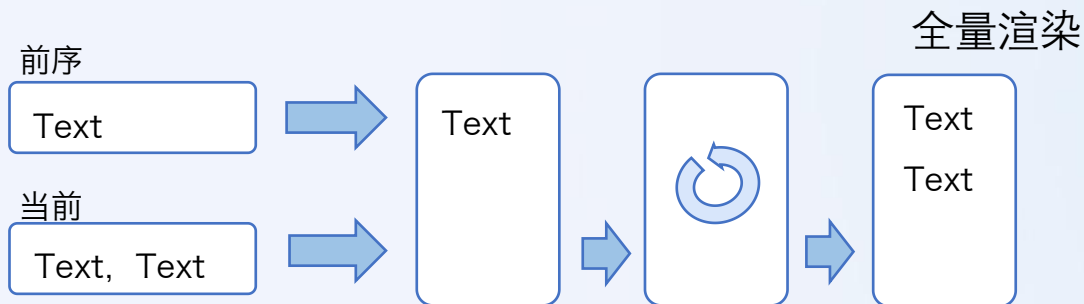


不完整的JSON



通过fixJson闭合JSON

实现局部增量渲染



通过diff-patch实现仅追加变更集，维持内存稳定



减少内存占用

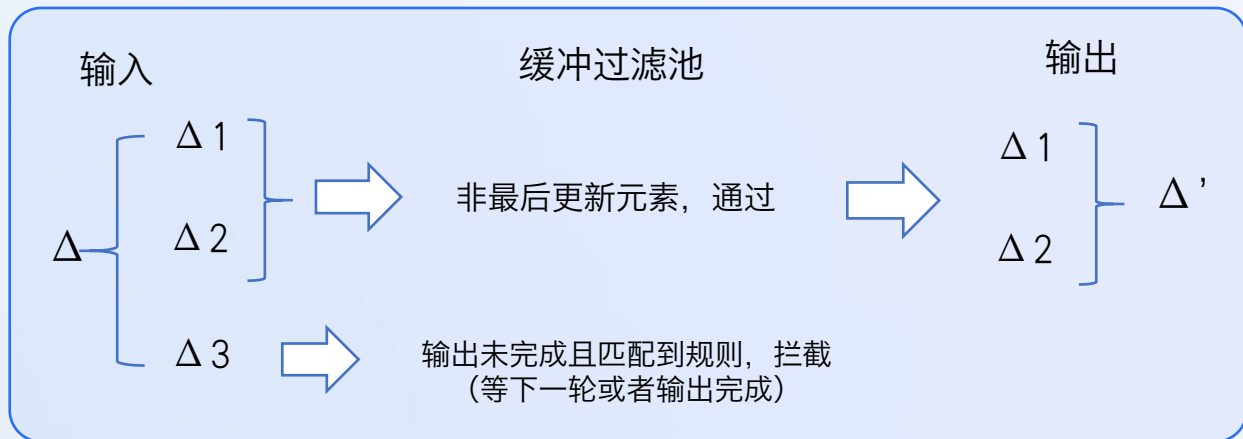


避免全量重绘



复杂场景高性能

原理：缓冲保护区



缓冲保护区简化原理图：削减不完整信息引起的错误与不必要的渲染

缓冲保护区核心机制：

从 Delta 变化集中获取当前最后一个变化中的值，

- 如果该值匹配到了缓冲规则，则拦截从 Delta 变化集删除该变化不体现，仅推送其他非最后的更新；
- 被删除的变化需要等待下一轮更新，当全部更新完毕或者变化值不是最后一条变化值时，可以推送该更新。

拦截异常信息 • 保障渲染稳定 • 降低系统开销

无缓冲渲染场景

1 信息展示型字符串

```
{"componentName": "Text", "props": { "text": "Hello World" }}
```

2 枚举类型字符串

```
{"componentName": "TinyButton"}
```

3 资源地址

```
{"componentName": "img", "props": { "src":  
  "http://example.com/foobar.png" }}
```

4 表达式/函数声明

```
{"type": "JSFunction", "value": "function()  
{ console.log('foo') }}"}
```

缓冲规则示例

1 componentName

2 [componentName=img] > props > src

3 [type=JSFunction] > value

采用JSON路径的selector语法，类似CSS selector语法

02

原理与场景落地

2 场地落地扩展

● 场景落地述求下的扩展



物料可定制扩展

支持品牌自定义组件和样式，让生成的UI更符合品牌形象，实现个性化视觉表达。



交互上下文扩展

能够理解并融合业务上下文信息，提供更智能的交互逻辑，提升用户操作的连贯性。



生成式 MiniApp

突破单一组件限制，从生成UI片段扩展到生成完整的小程序应用，覆盖全链路场景。

物料可定制扩展：更懂品牌与业务

企业可以将自己的品牌组件库接入系统，AI在生成UI时会优先使用这些品牌物料，确保视觉风格的统一性，让每一次生成都符合品牌规范。通过定制业务物料，解决复杂场景问题。

品牌调性

华为云

个人信息登记表

*

姓名

请输入姓名

*

邮箱

请输入邮箱地址

手机号

请输入手机号

性别

男

女

所在城市

请选择城市

兴趣爱好

阅读

运动

音乐

旅行

编程

订阅通知

提交

重置

内部系统

个人信息登记表

*

姓名

请输入姓名

*

邮箱

请输入邮箱地址

手机号

请输入手机号

性别

男

女

所在城市

请选择城市

兴趣爱好

阅读

运动

音乐

旅行

编程

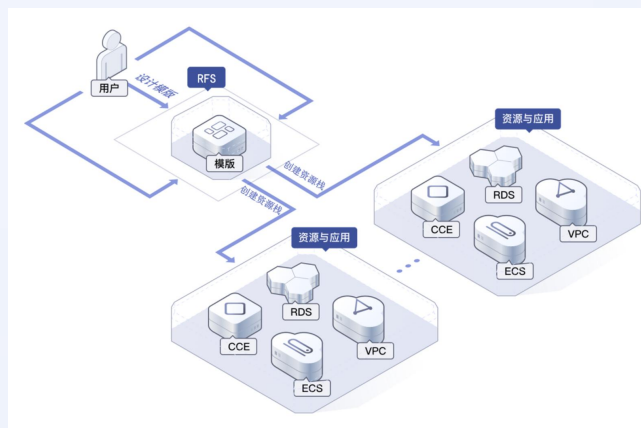
订阅通知

提交

重置

业务场景

拓扑图



员工接口搜索框

请选择审批人

lijing

李静 4109823

李菁菁 3506567

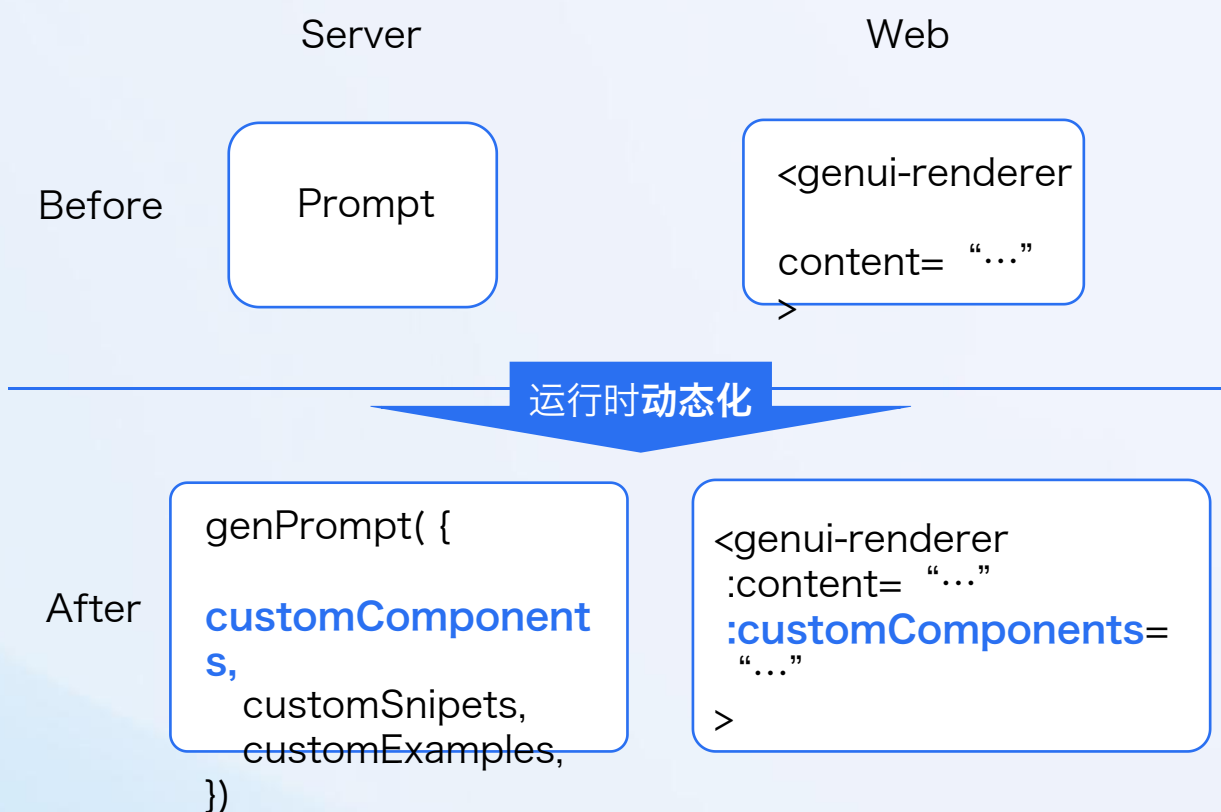
李静文 6676876

李敬文 7803245

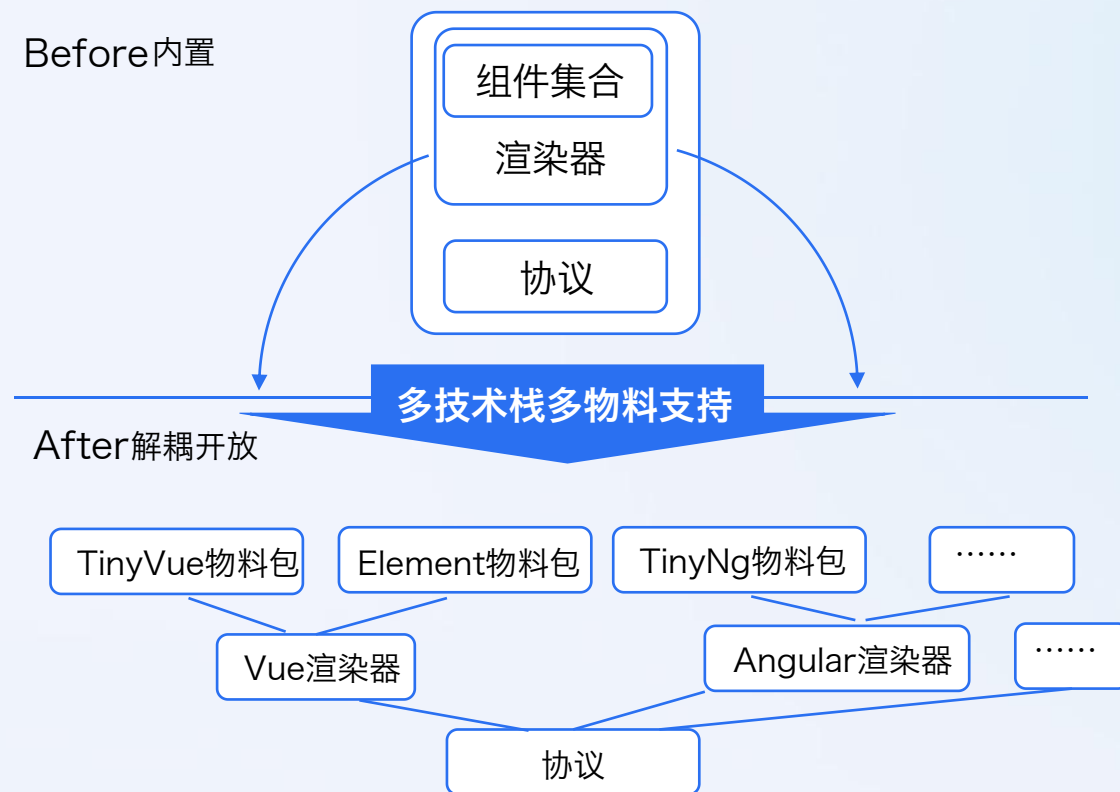
李景天 7937755

物料可定制可扩展，多技术栈支持

物料可定制扩展实现

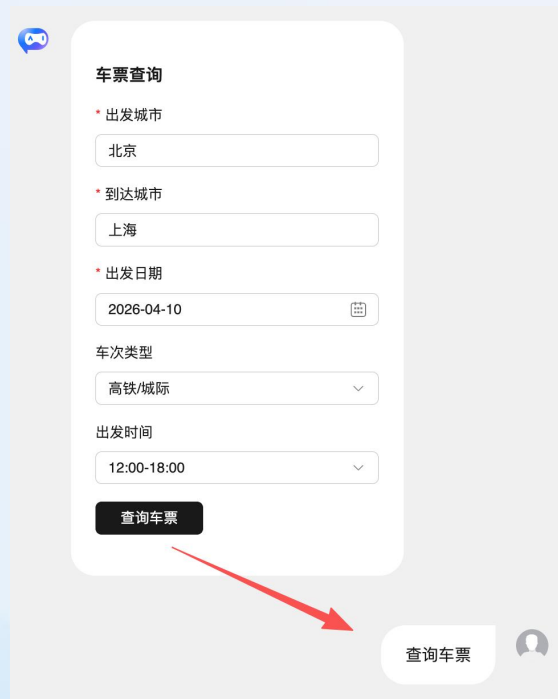


支持运行时动态追加组件



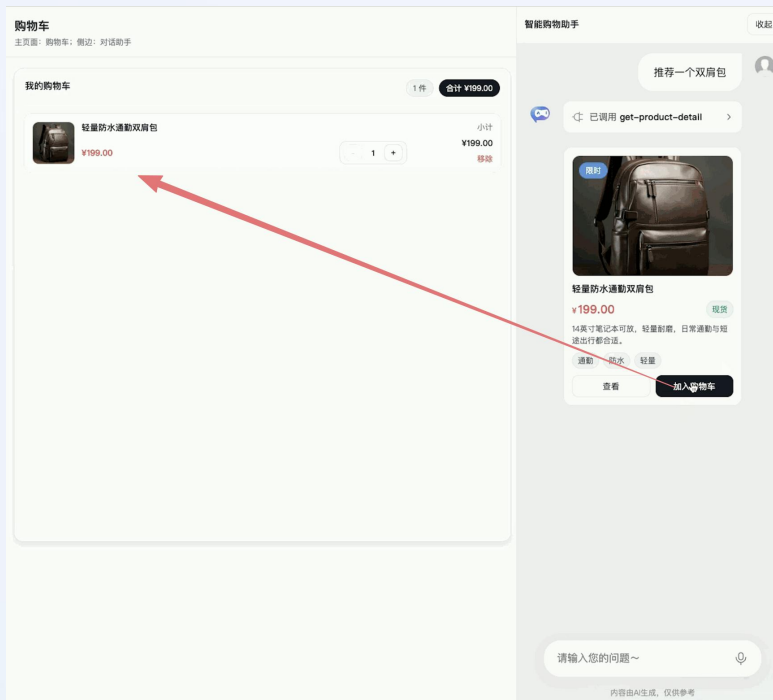
将协议、渲染器、物料包实现分层解耦

交互上下文扩展：跳出对话框，融合业务



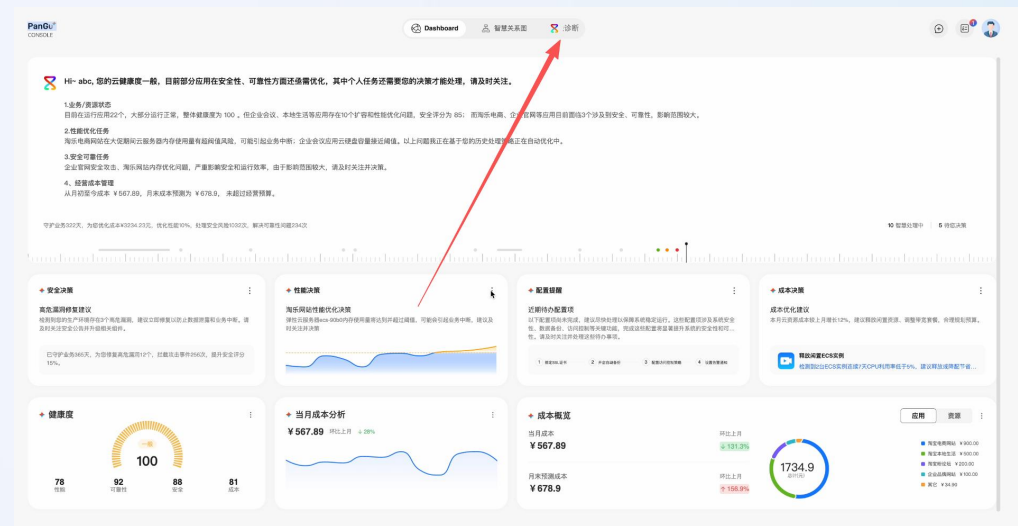
对话框内交互

支持通过卡片交互自动触发对话



对话框与对话框外的交互

支持在对话助手中的卡片交互触发页面的其他模块的功能



无对话框的交互

支持生成式内容与非生成式内容进行交互

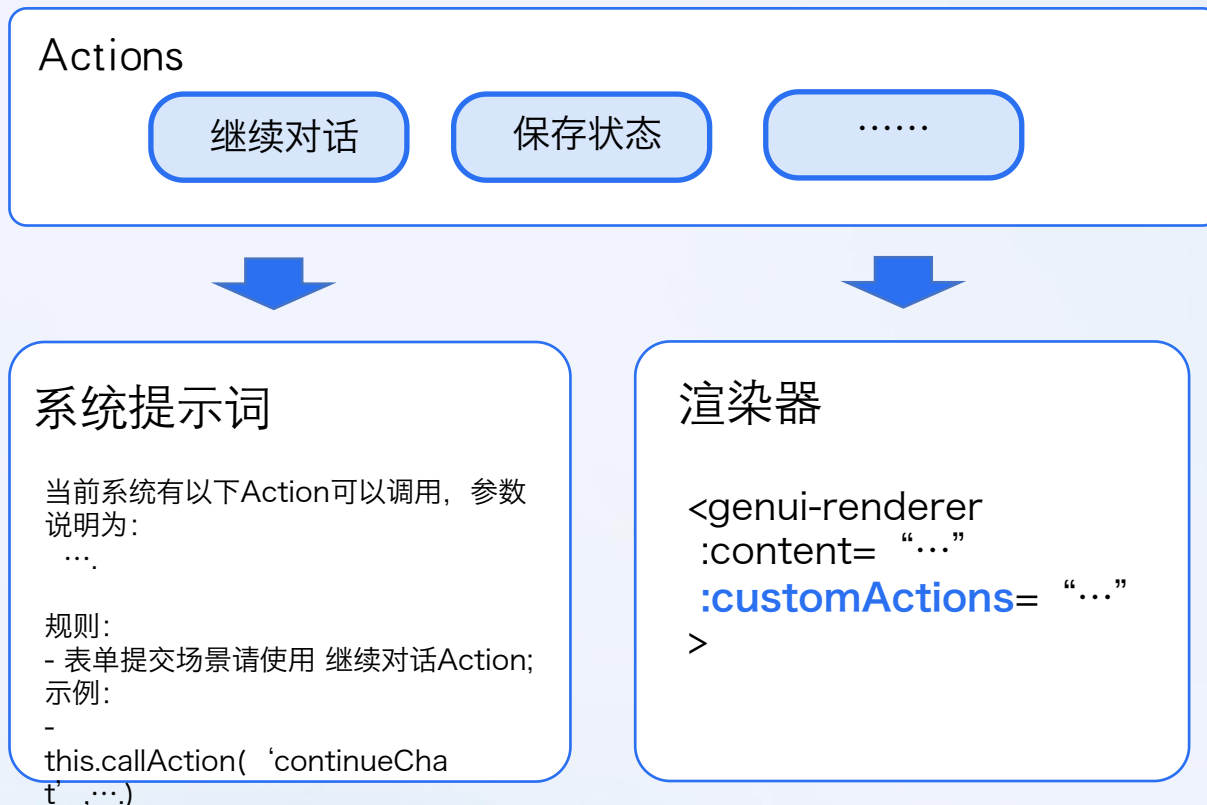
交互上下文扩展实现

初版实现：专门为表单提交场景触发继续对话设计的方案



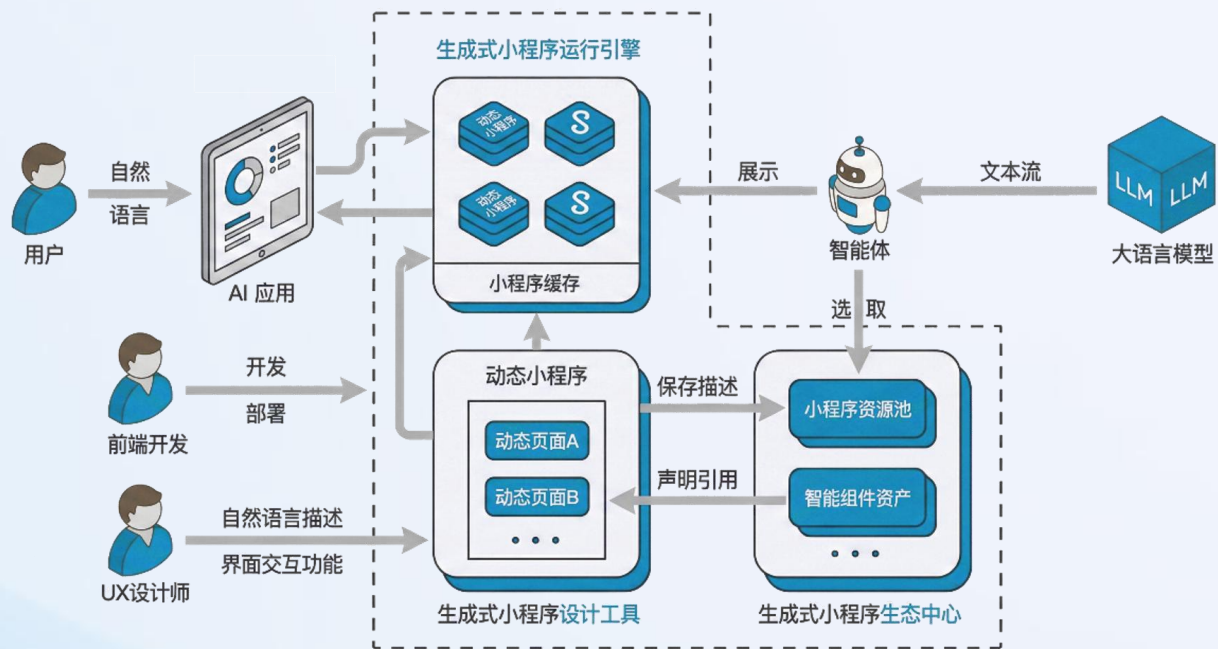
提供一个特殊的按钮组件，点击的时候自动触发上下文的继续对话函数调用

通用化实现：抽取Action，继续对话仅为其中一种Action

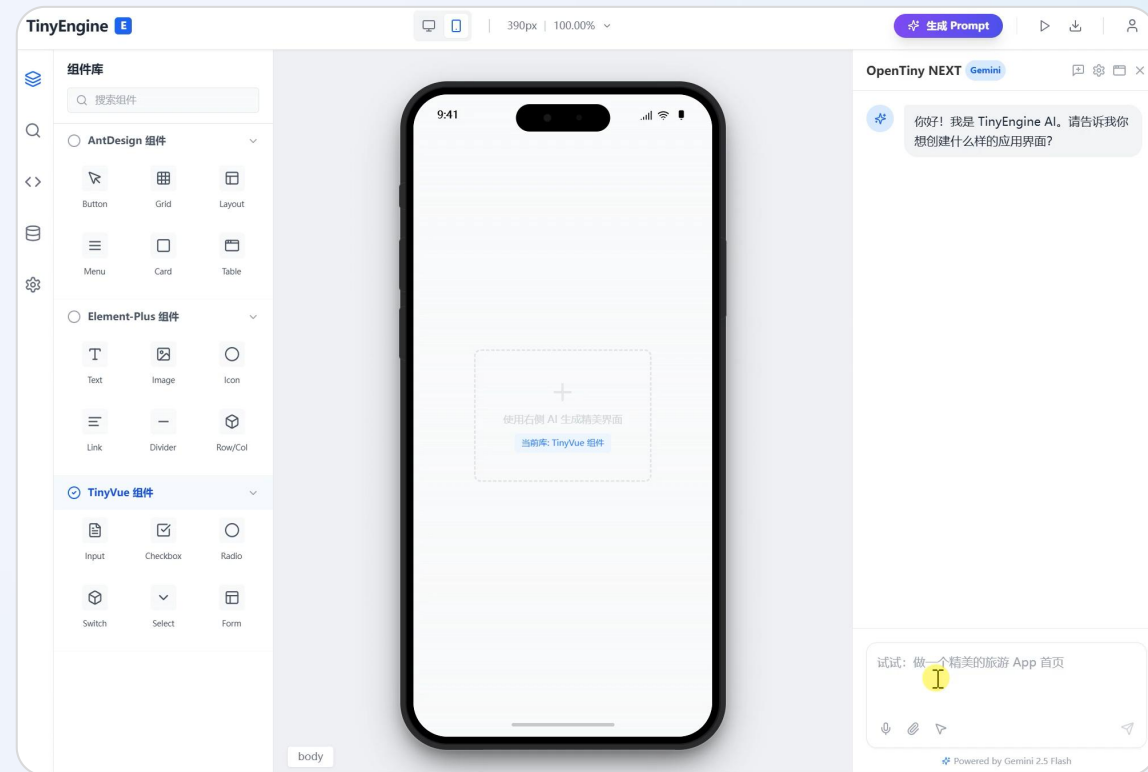


提供一个通用Action上下文工具，通过系统提示词告诉大模型有哪些Action，参数是什么，如何调用，并在渲染器完成调用

● 延长线：生成式 MiniApp， 共享对话生成应用



AI 应用平台与生成式小程序



对话生成应用

AI 驱动生成

⚡ 即时可用

🔗 一键分享协作



生成式MiniApp构建器



生成式MiniApp构建器 操作协议的优化分享 (树状结构适用)

JSON Diff Patch Delta 协议

协议简化描述

```
增: path: [ newValue ]
删: _path: [ originValue, 0, 0]
改: path: [newValue, originValue]
移: path: [", newIndex, oldIndex]
```

协议优势: 表达唯一

生成效果问题:

1. Path中数组下标识别糟糕

比如: children: { 3: [xxx]}, 长数组经常下标错误导致无法操作

2. 删除操作经常无法写对

比如: children: { _2: [xxx, 0, 0]}, 经常会写成 children: { 2: [xxx, 0, 0]}, 导致不能正确删除

RFC 6902 JSON Patch 协议

协议简化描述

```
增: { op: " add", path: " ", value: " " }
删: { op: " remove", path: " " }
改: { op: " replace", path: " ", value: " " }
移: { op: " move", from: " ", path: " " }
```

协议优势: 简单易懂

生成效果问题:

1. Path中数组下标识别糟糕

比如: {op: 'remove', path: '/children/0/children/2/children/3' }
长数组经常下标错误导致无法操作, 并且路径越长, 下标就越多出错

2. 连续操作后对下标影响难以修正

比如: 先删除数组第一个元素(index 0)再操作第三个元素 (index 2 -> 1), 输出为2导致连续操作失败

RFC 6902变体&强制ID索引

协议简化描述

```
增: { op: " add", id: " ", path: " ", value: " " }
删: { op: " remove", id: " ", path: " " }
改: { op: " replace", id: " ", path: " ", value: " " }
移: { op: " move", id: " ", positionId: " ", position: " ", before: " ", after: " ", append: " " }
```

协议优势: 通过增加id减少path长度, 且比较容易转换成RFC6902

生成效果改善:

1. 由于不依赖数组下标, 生成效果有较大幅度改善, 但是需要手动计算

遗留问题:

1. 大模型理解不正确的情况仍存在, 需要使用更简单的表述

比如: 交换位置1和位置3元素, 不如告诉到模型吧1移动到2后面, 把3移动到2前面

02

原理与场景落地

3 度量指标与局限性

AI 生成内容难保证，何时投放商用？

用户声音



大模型生成内容不稳定，有时候会输出不正确的内容，导致无法使用。

内容正确性



简短小卡片输出速度还可以，稍微长一点的内容生成很慢。

用户使用体验



这个系统提示词很费token，有没有办法优化？

经济与效益



为什么这个模型输出的时候好好的，换了另一家模型，这个输出就不对了。

大模型差异

生成式UI的度量指标：性能指标体系探索

 **核心思路：**借鉴大模型成熟的 TTFT/TPOT 指标体系，构建生成式UI的用户体验性能衡量标准。

关键指标一：首屏反馈时间

定义：用户从提交请求到看到第一个有效可视化反馈的时间。

意义：替代传统“三秒原则”，决定用户体验的第一印象。

关键指标二：渐进式渲染效率

定义：首个元素出现后，后续UI组件逐步呈现的节奏与间隔。

意义：衡量生成流畅度，类似大模型的Token流式输出体验。

优化策略

1. 尽可能优先输出有效的可视化的元素，或者引入一些骨架，让问答专注输出
2. 尽可能简化组件的使用配置项，把常用配置项作为默认值无需配置，增强局部片段引用等
3. 压缩系统提示（渐进式披露），压缩上下文对话（总结），保持上下文使用率中健康的空间



生成式UI的度量指标：信息表达能力



指标一：UI 的可用度

核心定义：评估生成的UI界面是否具备完成目标任务的实际能力。

评估维度：

- **完整性：**界面元素完整，无缺失或错误组件
- **功能性：**交互逻辑正常，按钮表单响应正确
- **信息充分性：**提供完成任务所需的全部关键信息

核心意义：确保UI不仅“像”，更要“能用”，是业务价值实现的基础。

可用度兜底策略

1. Zod校验确保UI可用，使用回落文本回复等方式兜底
2. 信息收集场景通过二次确认，确保信息无误再执行



指标二：Token效率

核心定义：衡量生成UI消耗的计算资源(Token)与其传递信息价值的比率。

评估维度：

- **信息密度：**对比纯文本，Token消耗是否合理
- **Token构成：**区分排版Token与核心内容Token的比例

核心意义：追求最经济的Token成本与最高效的信息传达，平衡体验与消耗。

Token消耗优化策略

1. 通过渐进式披露和搜索策略展示组件信息和示例
2. 提供一些快速美观排版组件减少大模型在样式token消耗
3. 避免长数据大模型复述



应用场景局限性



长数据列表展示

局限性：大模型逐行输出效率低，导致用户体验极差。

解法：

- **内置法：**直接推送数据，避免复述
- **编程法：**定义API，初始化自行调用



超复杂应用界面

局限性：逻辑复杂难以一次生成，上下文限制导致无法单次完成。

解法：

- **复杂度前移：**任务拆分与多轮迭代
- **片段引用：**引用已有片段，压缩输出



重复生成场景

局限性：重复生成浪费Token，且难以保证连续场景的一致性。

解法：

- **预生成：**存储常见UI，叠加修改
- **前序引用：**引用局部UI，追加微调

03

协议对比与标准化争论

03

协议对比与标准化争论

1 协议对比



生成式UI的范畴与技术流派



目前有很多“生成式UI（Generative UI）”框架，相似又有许多不同。
那么到底什么是生成式UI？这些框架又有什么区别呢？应该如何选择？

范畴思考：HTML/Markdown是否属于生成式UI？符合“声明式描述”与“流式渲染”特征。

广义定义：任何由AI生成、用于描述和渲染用户界面的结构化信息，均可纳入生成式UI范畴。



源码生成渲染型

代表：灵光闪应用

特点：大模型直接生成前端源码（如HTML、Vue、React），后端/浏览器端动态构建，最终在浏览器端渲染。

分析：发挥空间大但UI难约束；多不支持流式渲染，需等待完整编译。



声明式UI

代表：TinyGenUI, a2ui, cosui

特点：生成JSON等声明式描述，由特定渲染器解析。

分析：组件与配置受限，可控性强；流式渲染依赖解析器能力。



数据驱动UI更新

代表：agui

模式：UI界面预定义，模型仅推送状态变更数据。

特点：模型仅负责数据逻辑，不直接控制界面结构。

分析：灵活性低，但安全性高；UI变化被限制在预设范围内。

声明式协议对比

生成式UI 声明式协议对比			
对比维度	Tiny GenUI	A2UI	Cosui
UI 描述能力	通过树状数据结构嵌套； 仅支持深度优先先后渲染； 需要补充局部刷新操作协议	通过索引实现UI嵌套； 渲染顺序可以自定义 具备局部刷新能力	树状嵌套； 具备局部刷新能力
数据绑定形式	{ type: JSExpression, value: this.state.foo scope.foo }	/foo/bar (scope)	{{foo.bar}}
事件通信模式	JS方法 + 运行时自定义Action	仅通过Action与大模型对话	compile阶段传入自定义Action
安全约束	JS方法灵活度高，可能引入安全风险	后端接收用户操作行为描述，缺少及时反馈行为，需要通过大模型更新界面，	通过Action约束，较为安全，灵活度较低
物料扩展能力	支持现有组件库（Vue、Angular）	需要继承渲染器，需改造适配，存在一定的耦合	支持San框架的组件库



协议转换：A2UI协议 转 TinyGenUI协议

```
[
  {
    "beginRendering": {
      "surfaceId": "simple-input-button-surface",
      "root": "simple-row",
      "styles": {
        "font": "system-ui, -apple-system, sans-serif",
        "primaryColor": "#3b82f6"
      }
    },
    "surfaceUpdate": {
      "surfaceId": "simple-input-button-surface",
      "components": [
        {
          "id": "simple-row",
          "component": {
            "flow": {
              "children": {
                "explicitList": [
                  "simple-input",
                  "simple-button"
                ],
                "distribution": {
                  "start": "center",
                  "alignment": "center"
                }
              }
            }
          },
          "id": "simple-input",
          "component": {
            "TextField": {
              "label": {
                "literalString": "输入"
              },
              "text": {
                "path": "/inputValue"
              },
              "textFieldType": "shortText"
            }
          }
        }
      ]
    }
  }
]
```

```
{
  "id": "simple-button",
  "component": {
    "Button": {
      "child": "button-text",
      "primary": false,
      "action": {
        "name": "button_clicked",
        "context": [
          {
            "key": "value",
            "value": {
              "path": "/inputValue"
            }
          }
        ]
      }
    }
  },
  "id": "button-text",
  "component": {
    "Text": {
      "text": {
        "literalString": "按钮"
      },
      "usageHint": "body"
    }
  },
  "dataModelUpdate": {
    "surfaceId": "simple-input-button-surface",
    "contents": [
      {
        "key": "inputValue",
        "valueString": ""
      }
    ]
  }
}
```

UI声明转换

物料转换

数据转换

事件转换

```
{
  "componentName": "Page",
  "props": {
    "className": "simple-input-button-surface-page",
    "css": ".simple-input-button-surface-page {\n      font: system-ui, -apple-system, sans-serif;\n      primaryColor: #3b82f6;\n    }",
    "children": [
      {
        "id": "simple-row",
        "componentName": "div",
        "props": {
          "style": "display: flex; flex-direction: row; gap: 8px; justify-content: start; align-items: center;"
        },
        "children": [
          {
            "id": "simple-input",
            "componentName": "TinyInput",
            "props": {
              "label": "输入",
              "placeholder": "输入",
              "modelValue": {
                "type": "JSExpression",
                "value": "this.state.inputValue",
                "model": true
              }
            }
          }
        ]
      }
    ]
  }
}
```

```
{
  "id": "simple-button",
  "componentName": "TinyButton",
  "props": {
    "onClick": {
      "type": "JSFunction",
      "value": "function() {\n      this.callAction('continueChat', {\n        message: 'button_clicked' + JSON.stringify(['value', this.state.inputValue])\n      })\n    }",
    },
    "children": [
      {
        "id": "button-text",
        "componentName": "Text",
        "props": {
          "text": "按钮"
        }
      }
    ]
  },
  "state": {
    "inputValue": ""
  }
}
```

因TinyGenUI支持JS方法，A2UI协议抽象的方法定义和数据处理均可转换更底层JS函数处理。

03

协议对比与标准化争论

2 标准化争论

● 标准化争论焦点：是否只支持原生HTML元素？

在生成式UI的标准化讨论中，一个核心的争论点是：**标准化的UI描述协议是否应该只支持浏览器原生的HTML元素/一个固定子集**，还是应该允许扩展自定义的业务组件。



核心矛盾：通用性 (HTML) 与 业务表达力 (自定义组件) 的权衡



Generative UI 标准化，是否也要把UI组件定义标准化？然后映射到不同的OS开发框架的UI。



以Google A2UI为例，预定义组件是通过Catalog声明，A2UI中还是保留了自定义Catalog。



框架的作用是为了减少人类开发者的负担，但生成式UI当中已经不存在人类开发者这个角色，那么生成的结果为何要基于框架？



前端框架的作用不仅仅减少人类开发者的负担，也能降低AI的负担，一个复杂交互功能的MiniApp，AI从零写，token消耗大，错误率也会大大提升。



一旦支持定制，跨端跨系统就难适配

● 标准化争论焦点：是否只支持原生HTML元素？

正方观点：拥抱原生，保持标准简洁



浏览器内核相关标准应该与框架无关

标准化应专注于底层能力，避免与特定前端框架绑定，确保技术中立性。



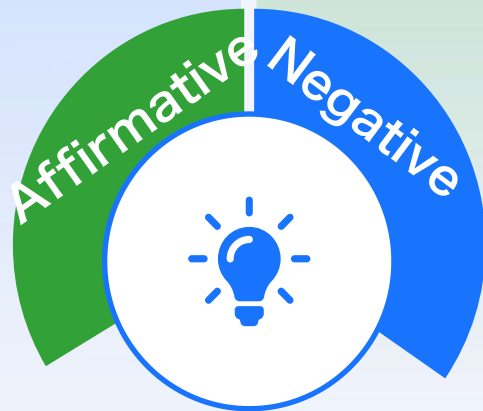
组件应当限制在通用范围

只支持原生HTML元素可以保证标准的通用性和稳定性，避免过度复杂导致的兼容问题。



标准化应围绕现有标准组件

基于现有的Web标准进行扩展，更容易被开发者广泛接受和浏览器厂商实现。



反方观点：开放扩展，满足业务需求



协议应保持开放可扩展

业务需求千变万化，需要协议具备足够的灵活性来支持自定义组件，避免被标准束缚。



业务侧有更多复杂诉求

复杂的业务场景需要专门的业务组件和上下文支持，原生元素无法满足深度业务逻辑。



仅有原生HTML元素效果差

仅使用原生HTML会导致AI生成速度慢、稳定性差，且难以满足严格的品牌视觉约束



案例分析：业务高级搜索框



高度复杂的交互逻辑

包含多条件筛选、动态加载选项及联动交互，非简单静态页面。



原生HTML构建的局限性

AI难以一次性生成正确的结构与逻辑，不仅效率低，且难以满足品牌视觉规范。



观点支撑

此类复杂业务组件有力证明了“纯AI原生生成”在实际开发中存在显著短板。

服务器高级搜索

Q 购买日期

2020年1月1日 - 2024年12月31日

AND

Q 区域

华北-北京四 × 华南-广州一 × +1

AND

Q 配置类型

2U4G × 2U2G × +2

+ 添加条件

排除

Q 状态

关机 ×

+ 添加排除条件

清除

保存查询

搜索

业务高级搜索框 UI 界面

总结

生成式UI通过将AI的能力从文本生成扩展到界面生成，极大地提升了人机交互的效率和体验，是AI交互的重要演进方向。尽管在应用落地方面仍有挑战，但它的潜力巨大，值得我们持续探索和投入。

Github: [opentiny/genui-sdk](https://github.com/opentiny/genui-sdk)

公众号: OpenTiny

欢迎star与关注

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

